

根据某条件聚类最少交换次数

题目描述

给出数字 K，请输出所有结果小于 K 的整数组合到一起的最少交换次数。

组合一起是指满足条件的数字相邻，不要求相邻后在数组中的位置。

数据范围：

- 100 ≤ K ≤ 100
- 100 ≤ 数组中数值 ≤ 100

输入描述

第一行 **输入数组** ： 1 3 1 4 0

第二行输入 K 数值： 2

输出描述

第一行输出最少交换次数： 1

用例

输入	1 3 1 4 0 2
输出	1

说明	小于2的表达式是1 1 0, 共三种可能将所有符合要求数字组合一起, 最少交换1次。
----	--

题目解析

本题可以使用 滑动窗口 解题。

本题需要我们将输入数组中所有小于 k 的元素聚集一起, 即形成一个连续区间。

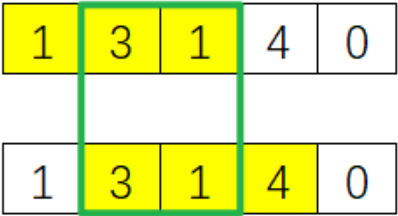
因此, 我们可以首先统计出输入数组中小于 k 的元素的总数, 将其作为滑窗的固定长度window_len, 即我们可以认为滑窗内部元素都是小于 k 的元素。

之后, 我们使用 window_len 长度滑动窗口在输入数组中滑动, 然后统计每一个滑动窗口内部有多少大于等于 k 的元素, 对于这样的元素, 我们需要交换出去。

因此滑动窗口内部有多少大于等于 k 的元素, 即代表对于状态的滑窗窗口需要交换多少次, 才能保证滑窗内部都是小于 k 的元素。

这里需要说明下, 我们不关心具体怎么交换的, 因为输入数组中小于 k 的 元素数量 时固定的, 假设滑窗内部有 x 个大于等于 k 的元素, 那么滑窗外部肯定就有对应的 x 个小于 k 的元素。

由于滑动窗口的移动是连续的, 因此对于相邻的两个滑窗, 是具有公共部分的, 比如下面绿色框:



CSDN @程序员阿甘

对于绿色框区域的元素, 我们完全没有统计两次的必要, 我们只需要统计滑动窗口每次移动后, 失去的元素和新增的元素即可。

1	3	1	4	0
---	---	---	---	---

①	3	1	④	0
---	---	---	---	---

失去

新增

CSDN @程序员阿甘

假设，第一个滑窗需要交换 $swap$ 次，才能保证内部元素都小于 k 。
看图，第二个滑窗 相较于 第一个滑窗，失去了**元素1**，新增了**元素4**

- 元素1小于 k ，因此失去元素1，第二个滑窗不会减少交换次数
- 元素4大于 k ，因此新增元素4，第二个滑窗需要增加交换次数

因此第二个滑窗需要 $swap + 1$ 次交换，才能保证内部元素都小于 k

基于这种仅计算相邻状态差异的策略，我们可以在 $O(n)$ 时间复杂度下完成所有滑窗内部交换次数的统计，最终保留最小交换次数即可。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const nums = (await readline()).split(" ").map(Number);
7      const k = parseInt(await readline());
8
9      // window_len: 所有小于k的元素个数
10     let window_len = nums.filter((num) => num < k).length;
```

运行

```

11 | 12 | // 记录结果
13 | let minSwap = Infinity;
14 |
15 | // 保证滑窗内都是小于k的元素, 所需的交换次数
16 | let swap = 0;
17 | for (let r = 0; r < nums.length; r++) {
18 |     // r 是滑窗的右边界, nums[r] 是滑窗新加入的元素
19 |     if (nums[r] >= k) {
20 |         // 如果nums[r] >= k, 则需要被交换, 交换次数+1
21 |         swap++;
22 |     }
23 |
24 |     // 当滑窗长度为 window_len + 1 时, 则滑窗左边界为 l = r - window_len
25 |     // 为了保证滑窗左边界为 window_len, 则此时左边界应该被丢弃, 即 nums[r - window_len] 应该被丢弃
26 |     // 如果 nums[r - window_len] >= k, 则滑窗内交换次数-1
27 |     if (r >= window_len && nums[r - window_len] >= k) {
28 |         swap--;
29 |     }
30 |
31 |     // 若 r < window_len - 1, 即使 l == 0, 滑窗长度也达不到 window_len
32 |     // 因此只有 r >= window_len - 1 时, 滑窗长度才能到达 window_len
33 |     if (r >= window_len - 1) {
34 |         minSwap = Math.min(minSwap, swap);
35 |     }
36 | }
37 |
38 | console.log(minSwap);
39 | })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |

```

```

4 | public class Main {
      5 |         public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int[] nums = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
9 |         int k = sc.nextInt();
10 |
11 |         // window_len: 所有小于k的元素个数
12 |         int window_len = 0;
13 |         for (int num : nums) {
14 |             if (num < k) {
15 |                 window_len++;
16 |             }
17 |         }
18 |
19 |         int minSwap = Integer.MAX_VALUE;
20 |
21 |         // 保证滑窗内都是小于k的元素, 所需的交换次数
22 |         int swap = 0;
23 |
24 |         for (int r = 0; r < nums.length; r++) {
25 |             // r 是滑窗的右边界, nums[r] 是滑窗新加入的元素
26 |             if (nums[r] >= k) {
27 |                 // 如果nums[r] >= k, 则需要被交换, 交换次数+1
28 |                 swap++;
29 |             }
30 |
31 |             // 当滑窗长度为 window_len + 1 时, 则滑窗左边界为 l = r - window_len
32 |             // 为了保证滑窗左边界为 window_len, 则此时左边界应该被丢弃, 即 nums[r - window_len] 应该被丢弃
33 |             // 如果 nums[r - window_len] >= k, 则滑窗内交换次数-1
34 |             if (r >= window_len && nums[r - window_len] >= k) {
35 |                 swap--;
36 |             }
37 |
38 |             // 若 r < window_len - 1, 即使 l == 0, 滑窗长度也达不到 window_len
39 |             // 因此只有 r >= window_len - 1 时, 滑窗长度才能到达 window_len
40 |             if (r >= window_len - 1) {

```

```

41         minSwap = Math.min(minSwap, swap);
42     }
43 }
44
45     System.out.println(minSwap);
46 }
47 }

```

Python算法源码

```

1  # 输入获取
2  nums = list(map(int, input().split()))
3  k = int(input())
4
5
6  # 核心代码
7  def solution():
8      # window_len: 所有小于k的元素个数
9      window_len = 0
10     for num in nums:
11         if num < k:
12             window_len += 1
13
14     minSwap = len(nums)
15
16     # 保证滑窗内都是小于k的元素，所需的交换次数
17     swap = 0
18     for r in range(len(nums)):
19         # r 是滑窗的右边界，nums[r] 是滑窗新加入的元素
20         if nums[r] >= k:
21             # 如果nums[r] >= k，则需要被交换，交换次数+1
22             swap += 1
23
24         # 当滑窗长度为 window_len + 1 时，则滑窗左边界为 l = r - window_len
25         # 为了保证滑窗左边界为 window_len，则此时左边界应该被丢弃，即 nums[r - window_len] 应该被丢弃

```

```

26 |         # 如果 nums[r - window_len] >= k, 则滑窗内交换次数-1
27 |         swap -= 1
28 |
29 |
30 |         # 若 r < window_len - 1, 即使 l == 0, 滑窗长度也达不到 window_len
31 |         # 因此只有 r >= window_len - 1 时, 滑窗长度才能到达 window_len
32 |         if r >= window_len - 1:
33 |             minSwap = min(minSwap, swap)
34 |
35 |     return minSwap
36 |
37 |
38 | # 输出打印
39 | print(solution())

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <math.h>
3 |
4 | #define MAX_SIZE 10000
5 |
6 | int main() {
7 |     int nums[MAX_SIZE];
8 |     int nums_size = 0;
9 |
10 |    while (scanf("%d", &nums[nums_size++])) {
11 |        if (getchar() != ' ') break;
12 |    }
13 |
14 |    int k;
15 |    scanf("%d", &k);
16 |
17 |    // window_len: 所有小于k的元素个数
18 |    int window_len = 0;

```

```
19     for (int i = 0; i < nums_size; i++) {20         if (nums[i] < k) {
21         window_len++;
22     }
23 }
24
25 // 记录结果
26 int minSwap = nums_size;
27
28 // 保证滑窗内都是小于k的元素，所需的交换次数
29 int swap = 0;
30 for (int r = 0; r < nums_size; r++) {
31     // r 是滑窗的右边界，nums[r] 是滑窗新加入的元素
32     if (nums[r] >= k) {
33         // 如果nums[r] >= k，则需要被交换，交换次数+1
34         swap++;
35     }
36
37     // 当滑窗长度为 window_len + 1 时，则滑窗左边界为 l = r - window_len
38     // 为了保证滑窗左边界为 window_len，则此时左边界应该被丢弃，即 nums[r - window_len] 应该被丢弃
39     // 如果 nums[r - window_len] >= k，则滑窗内交换次数-1
40     if (r >= window_len && nums[r - window_len] >= k) {
41         swap--;
42     }
43
44     // 若 r < window_len - 1，即使 l == 0，滑窗长度也达不到 window_len
45     // 因此只有 r >= window_len - 1 时，滑窗长度才能到达 window_len
46     if (r >= window_len - 1) {
47         minSwap = (int) fmin(minSwap, swap);
48     }
49 }
50
51 printf("%d", minSwap);
52
53 return 0;
54 }
```


C++ 算法源码

[复制](#)[运行](#)

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      vector<int> nums;
7
8      int num;
9      while (cin >> num) {
10         nums.emplace_back(num);
11         if (getchar() != ' ') break;
12     }
13
14     int k;
15     cin >> k;
16
17     // window_len: 所有小于k的元素个数
18     int window_len = count_if(nums.begin(), nums.end(), [&k](int num) { return num < k; });
19
20     // 记录结果
21     int minSwap = INT_MAX;
22
23     // 保证滑窗内都是小于k的元素, 所需的交换次数
24     int swap = 0;
25     for (int r = 0; r < nums.size(); r++) {
26         // r 是滑窗的右边界, nums[r] 是滑窗新加入的元素
27         if (nums[r] >= k) {
28             // 如果nums[r] >= k, 则需要被交换, 交换次数+1
29             swap++;
30         }
31
32         // 当滑窗长度为 window_len + 1 时, 则滑窗左边界为 l = r - window_len
33         // 为了保证滑窗左边界为 window_len, 则此时左边界应该被丢弃, 即 nums[r - window_len] 应该被丢弃
34         // 如果 nums[r - window_len] >= k, 则滑窗内交换次数-1
35         if (r >= window_len && nums[r - window_len] >= k) {
```

```
36         swap--;
37     }
38
39     // 若  $r < window\_Len - 1$ , 即使  $l == 0$ , 滑窗长度也达不到  $window\_Len$ 
40     // 因此只有  $r \geq window\_Len - 1$  时, 滑窗长度才能到达  $window\_Len$ 
41     if (r >= window_len - 1) {
42         minSwap = min(minSwap, swap);
43     }
44 }
45
46 cout << minSwap << endl;
47
48 return 0;
49 }
```

,